MULTI MODAL SMART VEHICLE

A Project Report

Submitted in partial fulfillment of the

Requirements for the award of the Degree of

BACHELOR OF SCIENCE (INFORMATION TECHNOLOGY)

By

Shekhar Suman

Seat No: - 3011075

Under the esteemed guidance of

Mr. Ameya Rane

(Lecturer)

DEPARTMENT OF INFORMATION TECHNOLOGY

ST. ROCK'S COLLEGE OF ARTS, COMMERCE AND SCIENCE

(Affiliated to University of Mumbai)

MUMBAI- 400092

MAHARASHTRA

2025-2026

ST. ROCK'S COLLEGE OF ARTS, COMMERCE AND SCIENCE

(Affiliated to University of Mumbai)

MUMBAI-MAHARASHTRA-400092

DEPARTMENT OF INFORMATION TECHNOLOGY

CERTIFICATE

This is to certify that the project entitled, "**Multi Modal Smart Vehicle**", is bonafied work of **SHEKHAR SUMAN** bearing Seat No: **3011075** submitted in partial fulfillment of the requirements for the award of degree of BACHELOR OF SCIENCE in INFORMATION TECHNOLOGY from University of Mumbai.

Internal Guide **Coordinator**

External Examiner

Date: **College Seal**

ABSTRACT

The **Multi-Modal Smart Vehicle** is a prototype designed to bridge the gap between manual driving and autonomous navigation by integrating user-centric controls with automated safety features. Built on an ESP32 architecture, the project aims to assist drivers in complex scenarios, such as navigating tight parking spaces via a 'Follow Mode' or preventing accidents through 'Autonomous Obstacle Avoidance'. The vehicle features a unique mechanical key-start ignition, a 3S Lithium-ion battery with BMS. Main achievements include the successful implementation of three operational modes—Manual, Obstacle Avoidance, and Following—and a complete automotive signalling system. Testing confirmed stable Wi-Fi-based remote control and improved path-redirection accuracy scanning. This project provides a scalable platform for future AI-driven enhancements, including voice recognition and collaborative semi-autonomous steering.

ACKNOWLEDGEMENT

I would like to express my deepest gratitude to everyone who contributed to the successful completion and implementation of the **Multi-modal Smart Vehicle** project. The journey of building this prototype, from initial circuit design to final multi-modal testing, has been an enriching experience made possible by the unwavering support of several individuals. First and foremost, I am profoundly thankful to **Ms. Palak Agrawat (HOD)** for her invaluable leadership and for providing the academic platform necessary to execute this project. Her vision and management of the department have been a constant source of inspiration throughout this academic endeavour.

I extend my sincere thanks to my project guide, **Mr. Ameya Rane (Lecturer)**, for his technical expertise, constant mentorship, and encouragement. His deep insights into ESP32 architecture and autonomous systems were crucial in overcoming complex challenges, particularly during the integration of obstacle avoidance logic and PWM speed optimization.

I am also grateful to my fellow students and peers who offered their suggestions and feedback during the project's various stages. Their collective wisdom and collaborative spirit helped transform a conceptual idea into a functional, multi-modal smart vehicle, and I appreciate the technical discussions that refined the system's logic.

Finally, I am incredibly thankful to my family and friends for their unwavering support and patience, which served as a constant source of motivation. Their encouragement was vital during intensive hardware debugging phases, such as resolving sensor synchronization issues and perfecting the unique key-start ignition system that defines this project.

DECLARATION

I hereby declare that the project entitled, "**Multi-modal Smart Vehicle**" done at Mumbai, has not been in any case duplicated to submit to any other university for the award of any degree. To the best of my knowledge other than me, no one has submitted to any other university.

The project is done in partial fulfillment of the requirements for the award of degree of **BACHELOR OF SCIENCE (INFORMATION TECHNOLOGY)** to be submitted as final semester project as part of our curriculum

Shekhar Suman

TABLE OF CONTENTS

Introduction **1**
1.1 Background 1
1.2 Aim and Objectives 1
1.3 Purpose, Scope and Applicability 2
1.3.1 Purpose 2
1.3.2 Scope 3
1.3.3 Applicability 4
Survey of Technologies **5**
2.1 Existing System 5
2.2 Proposed System 5
2.3 Justification for Selected Technology 6
Requirements and Analysis **7**
3.1 Problem Definition 7
3.2 Requirement Specifiction 7
3.2.1 Functional Requirements 7
3.2.2 Non Functional Requirements 8
3.3 Planning and Scheduling 8
3.3.1 Gantt Chart 9
3.3.2 Pert Chart 10
3.4 Software Requirements and Hardware Requirements 11
3.4.1 Software Requirements 11
3.4.2 Hardware Requirements 11
3.5 Conceptual Model 13
3.5.1 Block Diagram 13
3.5.2 Class Diagram 13
3.5.3 Flow Chart 14
3.5.4 Circuit Diagram 15
System Design **16**
4.1 Basic Modules 16
4.2 Procedural Design 17
4.2.1 The System Startup Procedure 17
4.2.2 The Main Executive Loop 17
4.2.3 The Fault Tolerance Procedure 18

4.3 Algorithm Design 18
4.3.1 Autonomous Obstacle Avoidance Algorithm 18
4.3.2 Follow Mode Algorithm 19
4.3.3 Manual Control & Interrupt Algorithm 19
4.4 User Interface (UI) Design 20
4.5 Test Cases Design 21
4.5.1 Hardware and Power Sequence Testing 21
4.5.2 Functional Logic and Navigation Testing 21
4.5.3 Signalling and Fail-Safe Testing 21
Implementation and Testing 23
5.1 Implementation Approaches 23
5.2 Code 24
5.3 Testing Approach 27
5.3.1 Unit Testing 27
5.3.2 Integrated Testing 28
5.4 Modifications and Improvements 28
Results and Discussion 30
6.1 Test Reports and Analysis 30
6.2 User Documentation (Product Manual) 31
Conclusions 32
7.1 Conclusion 33
7.2 Limitations of the System 33
7.3 Advantages of the System 33
7.4 Future Scope of the Project 34
References 35

TABLE OF FIGURES

Figure 1 : Gantt chart 1 9
Figure 2 : Pert chart 10
Figure 3 : Block Diagram 13
Figure 4 : Class Diagram 13
Figure 5 : Flow Chart 14
Figure 6 : Circuit Diagram 15

Introduction

1.1 Background

The rapid evolution of the automotive industry is currently cantered on the transition from conventional manual driving to sophisticated autonomous systems. Traditionally, vehicles relied entirely on human perception and mechanical control; however, the integration of microcontrollers like the ESP32 has made it possible to embed "intelligence" into small-scale prototypes. Existing work in this field often ranges from basic, fixed-sensor obstacle avoidance toys to high-end, expensive autopilot systems found in commercial electric vehicles. There remains a significant middle ground for versatile, "multi-modal" platforms that allow a user to switch between manual remote operation and assisted autonomous modes depending on the environment.

This project explores the synergy between wireless IoT (Internet of Things) communication and real-time sensor processing. By utilizing the Arduino IDE to program the ESP32, this vehicle implements a web-based dashboard that mimics modern "connected car" ecosystems. Unlike standard RC cars that offer only direct steering, this project incorporates an active scanning mechanism using a ultrasonic sensor. This approach is inspired by LIDAR technology used in full-sized autonomous cars, allowing the vehicle to "look" left and right to evaluate the safest path rather than just stopping when an obstacle is detected.

Furthermore, the context of this project is rooted in solving practical, everyday problems. Tight urban parking, narrow garage exits, and the high risk of minor collisions for beginner drivers are the primary motivations. By integrating a "Follow Mode," the vehicle can be led out of a parking spot like a digital valet, while the "Obstacle Avoidance" mode acts as an active safety layer. This project does not just aim to build a remote-controlled car; it aims to prototype a safety-first vehicle architecture that includes a complete automotive signalling system, including brake lights and turn indicators, to simulate a real-world driving environment.

1.2 Aim and Objectives

The primary aim of this project is to design and implement a Multi-modal Smart Vehicle that integrates manual wireless control with autonomous driver-assist features to enhance safety and maneuverability in complex environments. By leveraging the ESP32 microcontroller and the Arduino IDE, the project seeks to prototype a low-cost, scalable architecture that mimics high-end automotive safety systems. The ultimate goal is to create a vehicle that can reduce human error during tight-space navigation and provide a versatile platform for future AI.

Objectives:

To achieve the aforementioned aim, the following technical objectives were established and successfully executed:

1. **Hardware Integration:** To assemble a four-wheeled robotic chassis powered by a 3S Lithium-ion battery with a BMS, utilizing an L298N driver for motor control and a Buck Converter for stable 5V power distribution.
2. **Ignition System Design:** To implement a unique mechanical key-start ignition system with persistence logic, ensuring that the system remains operational once triggered until a manual shutdown is performed.
3. **Manual Control Interface:** To develop a WiFi-based Web Dashboard hosted on the ESP32, allowing real-time remote operation of the vehicle through a mobile or laptop browser without the need for external apps.
4. **Autonomous Navigation Logic:** To program a Obstacle Avoidance mode that utilizes an ultrasonic sensor to scan the environment (Left/Right) and calculate the optimal path to prevent collisions.
5. **Distance-Adaptive Tracking:** To create a Follow Mode using non-blocking C++ algorithms that allow the vehicle to maintain a constant distance from a user, moving forward or backward based on real-time sensor feedback.
6. **Visual Feedback & Signalling:** To integrate a 20x4 I2C LCD for status monitoring and a full automotive LED system, including headlights, brake lights, and synchronized blinkers for turn signals and hazards.
7. **Software Debugging:** To utilize the Arduino IDE Serial Monitor for troubleshooting sensor latency and optimizing PWM (Pulse Width Modulation) frequencies to ensure smooth motor startup and speed control.

1.3 Purpose, scope and Applicability

1.3.1 Purpose

The primary purpose of the Multi-Modal Smart Vehicle is to demonstrate how low-cost embedded systems can drastically improve vehicle safety and maneuverability. By moving away from a single-mode design, this project proves that a vehicle can adapt its behavior—whether being driven manually via a smartphone or navigating autonomously in a crowded room. It is being developed to reduce the cognitive load on drivers in high-stress situations, such as tight parking, and to provide a theoretical framework for "Shared Autonomy" where the machine assists the human rather than replacing them.

1. **Demonstrating Seamless Multi-Modal Control**: The three ways to control the robot are: automatic safety, where it uses sensors to instantly stop and avoid crashes; direct control, where a person steers it precisely using a phone app; and voice control, which allows for easy, hands-free operation using simple spoken commands.
2. **Advancing Human-Machine Interaction (HMI) and Usability:** This project has two key purposes: it functions as an effective teaching tool for students to easily learn about how integrated control systems and sensors work together. More importantly, it acts as a proof-of-concept for developing future smart robots for home assistance and automation, proving that complex tasks require a high degree of safety, flexibility, and intuitive user commands.
3. **Promote Engagement**: Foster a more interactive and engaged learning environment through the use of discussion forums, where students can participate in academic discussions, collaborate with peers, and seek assistance from faculty.
4. **Enhance Accessibility**: Offer a web-based solution that can be accessed from anywhere, allowing students and faculty to stay connected and manage academic tasks regardless of their location.

1.3.2 Scope

The scope of this project encompasses the complete engineering lifecycle of a four-wheeled robotic platform, including its structural design, hardware assembly, and specialized software programming. The technical methodology utilizes a non-blocking C++ architecture within the Arduino IDE, which allows the ESP32 to manage multiple concurrent tasks such as maintaining a localized WiFi Access Point, hosting an asynchronous Web Server, and processing real-time ultrasonic pings while simultaneously controlling motor PWM signals. Within this scope, the project specifically addresses the optimization of wireless command latency and the implementation of accurate, real-time distance mapping for autonomous navigation. However, the system is designed under the assumption of operation on relatively flat, indoor surfaces to maintain optimal sensor accuracy, and its current limitations include a lack of lateral or sideways movement in Follow Mode and a dependency on a direct WiFi line-of-sight for the manual dashboard interface.

1.3.3 Applicability

The Multi-Modal Smart Vehicle is designed with a versatile architecture that extends its utility across practical, academic, and social domains. The vehicle acts as a "Smart Valet" where users can lead the car out of difficult parking spots using Follow Mode or use the manual mode for remote delivery of small payloads. In the computer science world, this project serves as a comprehensive case study for real-time operating logic, I2C protocol implementation, and secure web-based hardware control. It provides a foundation for assistive technologies for the elderly or individuals with limited mobility, allowing them to control a transport vehicle with simple web-based gestures or automated "follow" commands.

Survey of Technologies

2.1 Existing System

The historical development of small-scale autonomous vehicles has progressed through several technological phases, each revealing specific constraints that the Multi-Modal Smart Vehicle seeks to overcome. Traditional toy-grade remote-controlled systems, typically utilizing 27MHz or 2.4GHz Radio Frequency (RF) circuits, represent a "dumb" architecture. While reliable for basic directional movement, these systems lack any onboard intelligence or sensory processing capabilities. Consequently, they cannot "see" obstacles or interact with their environment, nor do they provide a feedback loop to the operator regarding critical telemetry such as real-time battery percentages or precise distances to surrounding objects.

Modern hobbyist approaches often transition to the Standard Arduino-Bluetooth framework, usually pairing an Arduino Uno with an HC-05 module. However, this ecosystem remains limited by a notoriously short communication range and a "closed" software environment that forces users to rely on specific, non-customizable mobile applications. From a processing standpoint, the Arduino Uno's 8-bit architecture is frequently overtaxed when attempting to manage high-speed web hosting and complex real-time ultrasonic scanning simultaneously. This leads to latency issues and a lack of the fluid multitasking required for sophisticated autonomous navigation.

Furthermore, many existing autonomous rovers suffer from inefficient "static" sensing configurations. Because these vehicles utilize fixed forward-facing sensors, the entire chassis must rotate to scan the environment whenever an obstacle is detected. This repetitive "Stop-Start" motion not only leads to excessive mechanical wear on the DC gear motors but also results in significant battery drain. By analyzing these collective limitations—from the lack of feedback in RF systems to the processing bottlenecks of 8-bit microcontrollers and the mechanical inefficiency of static rovers—the necessity for a more integrated, dual-core, and servo-scanning solution becomes clear.

2.2 Proposed System (Multi Modal Smart Vehicle)

The proposed system introduces a "Connected Architecture" that treats the vehicle as an IoT (Internet of Things) node.

1. **Unified Control via Web Server:** By utilizing the ESP32's built-in WiFi, the proposed system hosts an asynchronous web server. This allows for a "Zero-Install" control interface where any device with a web browser act as the remote.

2. **Safety-First Hardware:** The proposed system includes an automotive-style Key Ignition System. This adds a physical security layer and ensures that the high-power 3S Li-ion battery is only engaged when the key is turned.

2.3 Justification for Selected Technology

The selection of the **ESP32 Microcontroller** and the **Arduino IDE** environment was a strategic engineering decision based on the following technical justifications:

1. **Dual-Core Processing Efficiency:** The ESP32 features two processing cores (Protocol CPU and App CPU). In our project, this is critical because one core can handle the high-speed Wi-Fi maintenance and Web Server requests, while the second core focuses entirely on real-time sensor processing and motor PWM. This prevents "jitter" or lag when the car is navigating obstacles while simultaneously updating the Web Dashboard.
2. **Advanced PWM (Pulse Width Modulation) Resolution:** Unlike standard microcontrollers that have limited 8-bit PWM, the ESP32 allows for high-resolution PWM (up to 16-bit). This was essential for our **Multi-Modal Smart Vehicle** to achieve smooth startup torque for the 4WD motors. It allowed us to set a MIN_SPEED of 160 to overcome friction, ensuring the car doesn't "hum" but moves instantly when a command is given.
3. **Integrated Web-Server Capabilities:** The ESP32 has enough onboard Flash memory to store a complete HTML/CSS/JavaScript dashboard. This eliminates the need for an external SD card or app. By writing the code in the **Arduino IDE**, we utilized the WebServer.h library to create a responsive UI that works on any modern browser, fulfilling our goal of "Universal Control."
4. **I/O Pin Versatility & I2C Support:** The 38-pin ESP32 provided the perfect number of GPIOs to manage our extensive hardware list:
    - **I2C Pins (21, 22):** Used for the 20x4 LCD display to save wiring.
    - **Digital Pins:** Used for the specialized LED signaling system (Headlights, Indicators, Brake).
5. **Stability in Development (Arduino IDE):** Choosing the **Arduino IDE** as the development environment provided access to the "Serial Plotter" and "Serial Monitor," which were vital for debugging the **"DIST: NA"** sensor issues. The vast community support for Arduino libraries ensured that the integration between the Ultrasonic sensor, and Wi-Fi was seamless and stable.

Requirements and Analysis

3.1 Problem Definition

The central challenge of this project lies in the seamless integration of high-level automotive safety features into a low-cost, accessible robotic architecture, addressing the inherent limitations of standard vehicles that rely solely on human intuition. In high-risk, low-visibility, or confined environments, human drivers often suffer from "perception latency," where reaction lag can lead to collisions when obstacles suddenly appear. To solve this, the project decomposes the problem into the development of an autonomous braking system that continuously pings the environment to make critical safety decisions in less than 50 milliseconds, effectively eliminating the risk associated with human delay.

Furthermore, the project tackles the difficulty of maneuvering in "dead-zones," such as reversing through narrow corridors or exiting blind spots where ground-level obstacles are invisible to a seated driver. By implementing a sophisticated "Follow Mode," the vehicle can precisely track a human leader's movements, essentially utilizing the human as a "live guide" for safe passage through complex spaces. This is paired with a solution for "communication and control fragmentation," which traditionally forces users to rely on specialized RF hardware or device-specific mobile applications. By creating a localized, WiFi-based web ecosystem, the project ensures platform independence, allowing the vehicle to be operated from any browser-enabled device without the need for additional software.

Finally, the project addresses the critical sub-problem of hardware reliability and power integrity, which frequently causes hobbyist prototypes to fail during high-current demands. Many such systems suffer from voltage drops that trigger system resets; however, this architecture ensures industrial-grade reliability through the integration of a physical key-start ignition and a 3S Lithium-Ion Battery Management System (BMS). This robust power infrastructure allows the vehicle to handle the significant current required for 4WD operation while maintaining a stable logic supply for the ESP32, ensuring that the safety and communication layers remain functional under all operating conditions.

3.2 Requirements Specification

3.2.1 Functional Requirements

1. **Dynamic Multi-Modal Switching:** The system must allow real-time switching between Manual, Obstacle Avoidance, and Following modes without requiring a system reboot.

2. **Proportional Distance Tracking:** In Follow Mode, the system must utilize a mapping function to adjust motor speed based on distance. If the target is between 20cm and 50cm, the car stays still; if the target moves further, the car accelerates proportionally to catch up.
3. **Real-time Status Telemetry:** The system must push live data—including the current Operating Mode, the measured distance (in cm), and connection status—to both the physical 20x4 LCD and the remote Web Dashboard.
4. **Automotive Signalling Protocol:** Brake Logic: Red LEDs must trigger instantly when the PWM duty cycle for motors is set to zero.
    - **Turning Logic:** Yellow LEDs must blink in synchronization with the direction of the turn (Left/Right).
    - **Hazard Logic:** A specific software flag must allow all four indicators to blink simultaneously during a system alert or manual hazard toggle.

3.2.2 Non-Functional Requirements

1. **System Responsiveness:** The primary goal is to minimize end-to-end latency to below 100ms. This ensures that the time between a user clicking "Forward" on the web dashboard and the L298N driver engaging the motors is nearly instantaneous, allowing for precise manual control in confined or high-risk environments.
2. **Safety Persistence:** The project incorporates "Key-Start" logic to mimic a professional automotive experience. Once the ignition sequence is triggered, the system must remain powered even if the key is removed. A complete shutdown is only initiated through a deliberate turn to the "OFF" position, preventing accidental power loss during operation.
3. **Power Stability:** To protect the ESP32 microcontroller, the design utilizes a high-efficiency Buck Converter to maintain a constant 5V output. This regulation is vital to prevent system resets or brownouts caused by the significant voltage fluctuations that occur in the 12V battery during high-torque motor maneuvers.
4. **Sensory Accuracy:** The navigational integrity depends on the precision of the HC-SR04 ultrasonic sensor. The objective is to calibrate the "Sense-Think-Act" loop to provide real-time distance readings with an error margin of less than ±2cm, ensuring reliable autonomous braking and effective obstacle avoidance.

3.3 Planning and Scheduling

A well-structured planning and scheduling approach is essential for the successful implementation of the Arduino-based robotic project. This section outlines the phases, tasks, and timelines involved in the project development.

3.3.1 Gantt Chart

A Gantt chart provides a visual timeline for the tasks in the project, showing the planning and scheduling of entire project.

3.3.2 Pert Chart

A PERT (Program Evaluation and Review Technique) Chart helps visualize the project workflow, dependencies, and estimated completion times. Below is a breakdown of the project in terms of tasks and dependencies.

3.4 Software Requirements and Hardware Requirements

3.4.1 Software Requirements

The Software ecosystem for this project is built on the principle of open-source flexibility and real time performance.

1. **Operating System:** Windows 10/11 or macOS was utilized as the development host. These platforms provide the necessary USB drivers (CP210x or CH340) required for the PC to communicate with the ESP32 via a micro-USB or Type-C cable.
2. **Integrated Development Environment (IDE):** Arduino IDE (Version 2.3.x) served as the primary coding and flashing environment. It was chosen for its robust "Serial Monitor" (set to 115200 baud), which was essential for debugging real-time ultrasonic distance values and Wi-Fi connection logs.
3. **Programming Language:** The project is written in C++, utilizing the Arduino framework. This allows for low-level memory management and high-speed execution, which are critical for the non-blocking logic used in obstacle avoidance.
4. **Core Libraries:**
    - **WiFi.h:** Configures the ESP32 as a "Soft Access Point," allowing it to generate its own independent wireless network and host a localized web server for platform-independent control.
    - own network ("ESP") with a WPA2-encrypted password.
    - **WebServer.h**: Powers the internal dashboard, handling HTTP "GET" and "POST" requests from the user's browser to move the car or switch modes.
    - **LiquidCrystal_I2C.h:** Manages the communication with the LCD via the I2C protocol (SDA/SCL pins), reducing the number of required GPIO pins.
    - **Web Technologies:** The control interface was developed using HTML5 for structure, CSS3 for the "Joystick-style" UI, and JavaScript to send asynchronous commands to the ESP32 without refreshing the page.

3.4.2 Hardware Requirements

The hardware was selected to balance high power (for the 4WD drive) with delicate logic processing (for the sensors).

1. **Control Unit**
    - ESP32-WROOM-32: A 32-bit dual-core processor running at 240MHz. One core handles Wi-Fi/Web Server tasks, while the other handles real-time sensor interrupts.

- ESP32 Expansion Shield: This purple PCB provides dedicated 5V, GND, and Signal pins for every GPIO, preventing messy wiring and short circuits.

2. **Power & Regulation**
    - 3S Li-Ion Battery Pack: Three 18650 cells in series provide a nominal voltage of 11.1V and a peak of 12.6V. This high voltage is necessary to provide the torque required for the 4-motor chassis.
    - 3S BMS (Battery Management System): A critical safety component that prevents over-charging, over-discharging, and short-circuits.
    - LM2596 Buck Converter: Steps down the battery's 12V to a stable 5V/3A. This ensures the ESP32 and LCD receive clean power, even when the motors draw high current.
3. **Perception & Feedback**
    - HC-SR04 Ultrasonic Sensor: Measures distance by sending a 40kHz sonic burst and timing the echo. It is used for both the 20cm safety stop in Obstacle Mode and the tracking logic in Follow Mode.
    - 20x4 I2C LCD: Displays the system's "Heartbeat"—showing the current mode (MAN/OBS/FOL) and live distance readings.
    - Battery Level Indicator: A hardware bar-graph module that gives the user a visual "Fuel Gauge" of the battery's remaining capacity.
4. **Drive Train & Signalling**
    - L298N Dual H-Bridge: A high-power motor driver that regulates the speed (via PWM) and direction of the four DC motors.
    - Four High-Torque DC Gear Motors: These serve as the primary actuators for the 4WD chassis. The internal reduction gears provide the necessary torque to move the vehicle's weight and overcome surface friction, while the 1:48 gear ratio ensures a balanced speed-to-torque output for precise maneuvering.
    - Key Ignition Socket: A physical lock that acts as the master power trigger, providing a realistic automotive experience.
    - LED Signalling Suite: Includes 2x White (Headlights), 2x Red (Brake), and 4x Yellow (Indicators), wired to specific GPIOs for automated visual feedback.

3.5 Conceptual Model

3.5.1 Block Diagram

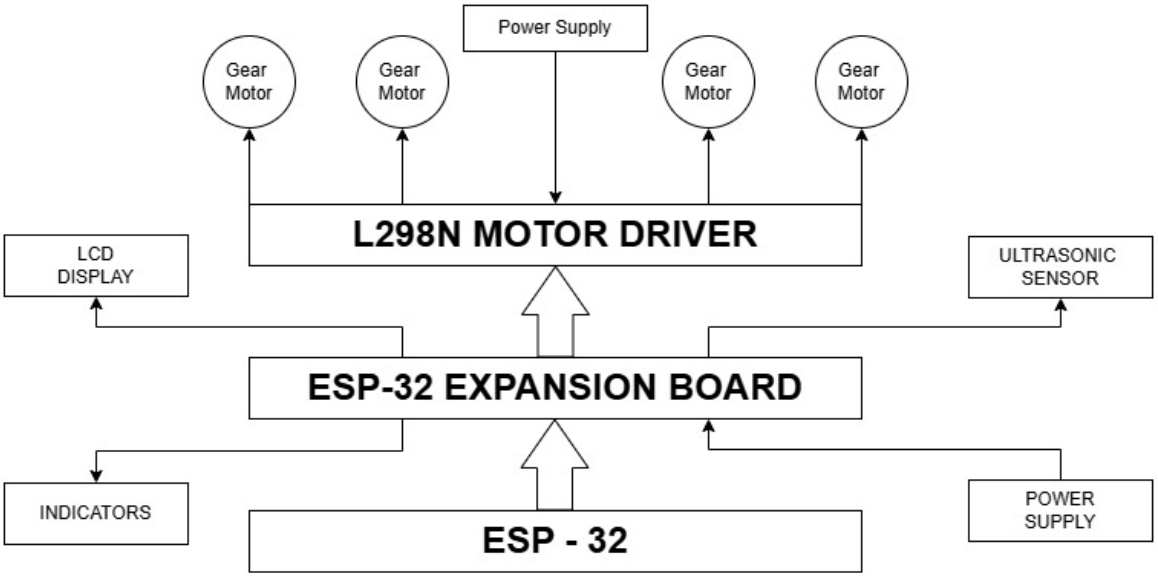


Figure 3: Block Diagram

3.5.2 Class Diagram

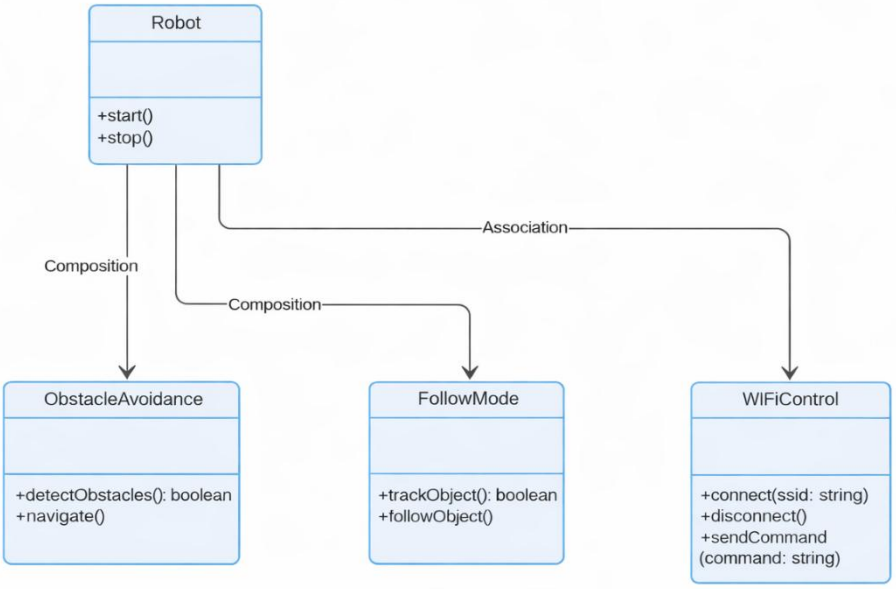


Figure 4: Class Diagram

3.5.3 Flow Chart

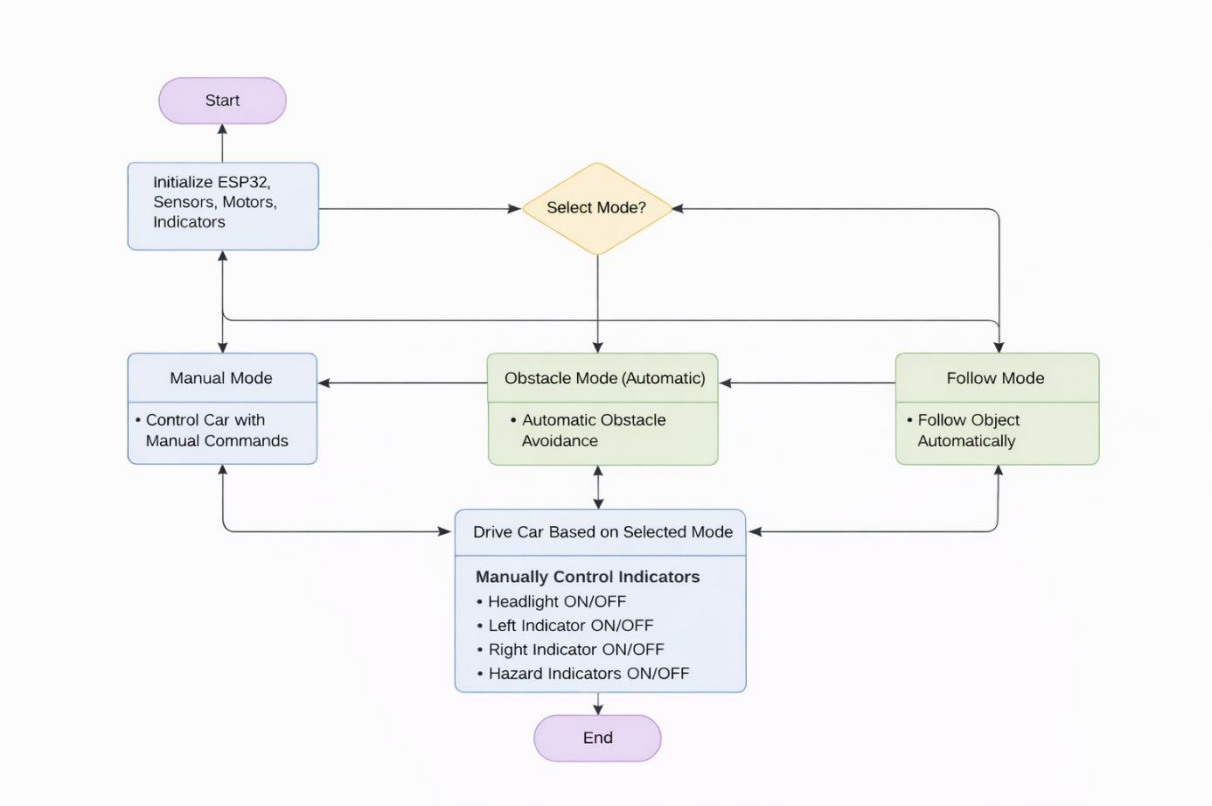


Figure 5: Flow Chart

3.5.4 Circuit Diagram

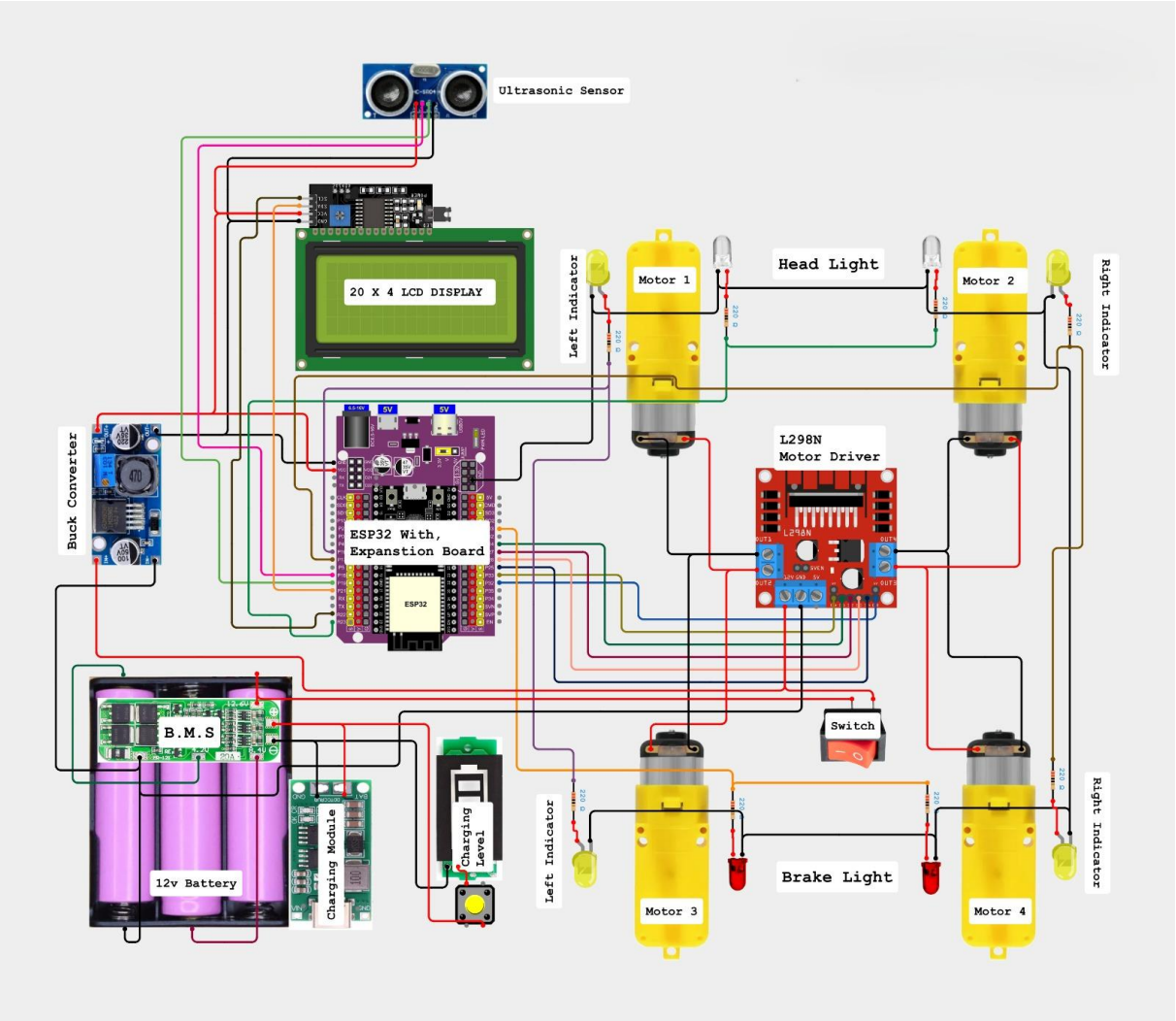


Figure 6: Circuit Diagram

System Design

4.1 Basic Modules

The system architecture of the Multi-Modal Smart Vehicle follows a strict "Divide and Conquer" methodology, ensuring that hardware and software components can be developed and debugged independently before final integration. At the forefront is the Wi-Fi Gateway and Web Hosting Module, which is responsible for initializing the ESP32's internal radio in Access Point (AP) mode. This module manages the SSID broadcasting and the WPA2 security handshake while serving a localized web dashboard. By using asynchronous HTTP handlers, the logic ensures the user interface remains responsive and lag-free, even during high-speed motor operations or complex sensor calculations.

The core decision-making is handled by the Navigation Intelligence and State Machine Module, which acts as the "Executive Controller" for the entire platform. This module manages the seamless transition between different operational modes by maintaining a global state variable. This logic dictates which specific algorithm—Manual, Obstacle Avoidance, or Following—takes priority over the GPIO pins at any given microsecond, ensuring that the vehicle's behavior is always predictable and safe regardless of user input or environmental changes.

Environmental awareness is provided by the Sensor Perception and Vision Module, which handles the precise low-level timing required for the HC-SR04 ultrasonic sensor. The underlying logic converts raw "echo" microseconds into a real-time distance measurement in centimeters using the speed of sound formula: Distance = (Time × 0.034) / 2. This data is then fed into the state machine, allowing the vehicle to "see" its surroundings and make autonomous braking or pathfinding decisions with a minimal error margin.

Physical movement is executed by the Drive Train and PWM Actuation Module, which directly interfaces with the L298N motor driver to control the 4WD chassis. To ensure smooth operation, the module utilizes the ESP32's ledc peripheral to generate high-frequency 20kHz PWM signals. This high-frequency approach is critical for providing sufficient startup torque to the four gear motors while simultaneously preventing the high-pitched "humming" noise often associated with lower-frequency motor control methods.

Finally, the Telemetry and Status Feedback Module orchestrates the I2C communication with the 16x2 LCD and the automation of the 6-LED signaling suite. This module serves as the vehicle's communication layer, mapping internal motor states—such as Forward,

Backward, and Stop—to physical light behaviors like headlights, brake lights, and blinkers. By simulating these real-world automotive standards, the system provides intuitive visual feedback to the operator, ensuring that the vehicle's current status and intended movements are always clearly visible.

4.2 Procedural Design

The procedural design of the Multi-Modal Smart Vehicle is built on a Non-Blocking Cyclic Executive model. Instead of using the standard delay() function, which freezes the processor, the system uses "polling" to check every component in a continuous loop. This ensures the car remains responsive to WiFi commands even while it is measuring distances or scanning for obstacles.

4.2.1 The System Startup Procedure

The operational procedure of the Multi-Modal Smart Vehicle begins the moment the physical key is engaged in the mechanical ignition socket, completing the primary power circuit. Upon receiving power, the ESP32 initiates a comprehensive hardware initialization sequence, clearing its internal registers and configuring the specific GPIO pins required to interface with the L298N motor driver and the integrated LED signaling suite. Simultaneously, the system performs a critical I2C handshake to establish stable communication with the liquid crystal display, which immediately provides user feedback by presenting a "System Initializing..." status message.

Once the hardware layer is stabilized, the software transitions into the network setup phase, where the internal Wi-Fi module is configured to operate in "Access Point" mode. In this state, the vehicle broadcasts its dedicated SSID, titled "ESP," and initializes an asynchronous internal web server at the static IP address of 192.168.4.1. The procedure concludes as the system enters a stable standby state; the onboard display updates to show "MODE: MANUAL," and the white headlights automatically illuminate. This visual and digital confirmation signifies that the initialization is complete and the vehicle is fully prepared to receive and execute user commands from the web dashboard.

4.2.2 The Main Executive Loop (Operational Phase)

Once the system is "ON," the ESP32 enters an infinite loop executing the following procedural steps in sequence:

1. **Wi-Fi Request Polling:** The system checks the WebServer.handleClient() function. If a user clicks a button on the Web Dashboard, the software parses the URL (e.g., /forward) and updates the Control Flag.

2. **Perception Trigger:** The software triggers the HC-SR04 "Trig" pin for 10 microseconds. It calculates the time for the "Echo" to return and updates the global distance variable.
3. **State-Dependent Decision Making:** The system checks which of the three modes is active:
    - **Manual Mode:** If the Manual flag is set, it executes movement commands directly from the Wi-Fi buffer.
    - **Obstacle Mode:** It compares the distance to the safety threshold (20cm). If the distance is lower, it halts the main loop to execute the Scanning Sub-procedure.
    - **Follow Mode:** It compares the distance to the target range (20cm - 50cm) and calculates the proportional motor speed.
4. **Actuator Update:** The calculated logic is converted into hardware signals. PWM values are sent to the motors, and digital signals are sent to the LEDs (e.g., Brake Lights turn ON if the car is stationary).
5. **Telemetry Feedback:** The current Mode, Distance, and Connection status are formatted into a string and sent to the I2C LCD for visual monitoring.

4.2.3 The Fault Tolerance Procedure (Safety Phase)

The procedural design incorporates "Interrupt-like" safety checks to manage unexpected events and ensure continuous operational integrity. This logic includes a Sensor Timeout protocol where, if the ultrasonic sensor fails to return a valid distance measurement (indicated as DIST: NA), the car is programmed to immediately halt all motors to prevent a blind collision. Complementing this is the Key Persistence Logic, which ensures that the procedural loop remains active even if the physical key is removed while the system is running; the system only triggers a formal Shutdown Procedure when the key is re-inserted and deliberately turned to the "OFF" position, providing a fail-safe against accidental power interruptions during a mission.

4.3 Algorithm Design

The intelligence of the vehicle is distributed across three primary algorithmic logic paths. Each algorithm is designed to be non-blocking, ensuring the ESP32 remains responsive to Wi-Fi interrupts while executing movement logic.

4.3.1 Autonomous Obstacle Avoidance Algorithm (The Scanning Logic)

The Obstacle Avoidance Algorithm is designed to mimic the human "stop-look-decide" process, utilizing real-time distance data from the HC-SR04 ultrasonic sensor to navigate

complex environments autonomously. During the logic process, the system maintains continuous monitoring by pinging the environment while moving forward; if the input data shows a distance falling below the 20cm detection threshold, the stopMotors() function is immediately triggered to prevent a collision. This safety halt is accompanied by brake signaling, where the Red LEDs are set to HIGH for visual feedback, followed by a comparative analysis where the car executes a timed turnLeft() if the left-side distance is greater than the right, or a turnRight() otherwise. The final output data consists of digital HIGH/LOW signals and PWM speed values sent to the L298N driver pins, effectively converting sensory perception into a reliable mechanical response.

4.3.2 Follow Mode Algorithm (The Proportional Tracker)

The Follow Mode operates on a sophisticated distance-based logic loop designed to maintain a consistent physical link between the vehicle and a target through real-time feedback. Utilizing input data from the HC-SR04 ultrasonic sensor, the algorithm establishes a "Neutral Zone" safety buffer between 20cm and 50cm, where the motors are set to 0 PWM to keep the vehicle stationary. If the target moves beyond the 50cm tracking threshold, the vehicle initiates forward movement using a proportional speed calculation; specifically, a mapping function translates distances between 50cm and 100cm into a PWM range of 160 to 255, causing the car to accelerate as the user moves further away. To prevent collisions, a backward safety logic is triggered if the distance drops below 20cm, forcing the vehicle to reverse at a fixed PWM of 180 to restore a safe gap. This process culminates in the output of dynamically updated PWM duty cycles (0–255) sent to the ledcWrite() function on the ESP32, ensuring a fluid and responsive tracking behavior.

4.3.3 Manual Control & Interrupt Algorithm

The Manual Control Logic serves as the primary interface between the user and the vehicle, utilizing a responsive Wi-Fi-based command structure to translate remote inputs into physical movement. The process begins with the reception of Input Data in the form of HTTP GET strings, such as "/F" for forward or "/S" for stop, which are sent via the localized web dashboard. During the Logic Process, the internal web server parses these characters and employs a switch-case statement to trigger the corresponding movement function, ensuring a high-speed transition between states. To maintain professional automotive standards, the algorithm performs Signal Synchronization, where triggering a "Left" or "Right" command simultaneously initiates the yellow LED pins in a blinking state; this is managed via a millis() timer to prevent blocking the main execution loop. Furthermore, the algorithm updates the

LCD Telemetry by formatting the current command into a readable string, such as "MOVING: LEFT," and pushing it to the I2C-connected display. This sequence results in high-priority Output Data, consisting of immediate motor state updates and LED status flags that ensure the vehicle is both maneuverable and visually communicative.

4.4 User Interface (UI) Design

The UI is designed following the "Mobile-First" philosophy, ensuring that the vehicle can be controlled smoothly from a smartphone touch-screen, which is the most common use case for a portable smart vehicle. The architecture is built using a Client-Server model where the ESP32 acts as the Web Server and the user's browser acts as the Client.

1. **External Components:** These are the visual elements the user see, such as the directional pad (D-pad), mode toggle buttons, and live telemetry labels.
2. **Internal Components:** This involves the Asynchronous JavaScript (AJAX) logic that runs in the background. When a button is pressed, JavaScript sends a background request to the ESP32 IP address without reloading the page, preventing any lag in motor response.
3. **User Analysis:** The interface is designed for both hobbyists and novice drivers. It uses intuitive icons (arrows for movement) and clear text labels for mode status.
4. **Task Analysis:** The primary tasks identified are movement control, switching autonomous modes, and monitoring safety data (distance).
5. **Environment Analysis:** Since the car is a mobile unit, the UI must work in outdoor or indoor settings via a local WiFi Access Point, meaning it does not require an active internet connection.
6. **Header & Status Zone:** Displays the project name: "Multi-Modal Smart Vehicle". Contains a real-time status bar showing the current mode (Manual, Obstacle, or Follow) and the distance to the nearest object.
7. **Control Zone (The Virtual Joystick):** Movement Buttons: A set of five large buttons arranged in a cross pattern for Forward, Backward, Left, Right, and Stop.
8. **Mode Selector:** A dropdown menu or a set of radio buttons that allow the user to switch the vehicle's "brain" between manual and autonomous logic.
9. **Auxiliary Feature Zone:** Lighting Controls: Toggles for the white Headlights and the Hazard blinker mode.
10. **Feedback:** A log area that shows "Command Received" confirmations to ensure the user knows the car is responding.

4.5 Test Cases Design

To ensure the reliability of the Multi-Modal Smart Vehicle, a comprehensive testing strategy was developed. This phase involves verifying the "Sense-Think-Act" cycle across different operational conditions to ensure that the hardware components and the C++ logic in the Arduino IDE work in perfect harmony.

4.5.1 Hardware and Power Sequence Testing

The first set of tests focuses on the physical integrity and the startup sequence of the vehicle.

1. **Cold Boot and Initialization Test:** This test ensures that the system triggers correctly upon turning the physical key. When the key is turned to "ON," the procedure must verify that the ESP32 powers up, the 20x4 LCD initializes with the correct welcome message, and the Wi-Fi Access Point ("ESP") becomes visible on external devices.
2. **Ignition Persistence Test:** A critical safety test where the physical key is removed while the system is running. The expected behavior is that the vehicle remains operational (Persistence Logic) and only shuts down when the key is re-inserted and turned to the "OFF" position.
3. **Voltage Stability Test:** This involves monitoring the LM2596 Buck Converter output. During high-torque maneuvers (like a 360-degree turn), the procedure verifies that the 5V rail remains stable for the ESP32 and sensors, even as the 12V battery current fluctuates.

4.5.2 Functional Logic and Navigation Testing

These tests verify the core algorithms governing the three operational modes.

1. **Obstacle Detection and Emergency Braking:** In Obstacle Avoidance Mode, the vehicle is driven toward a wall. The test is successful if the vehicle detects the obstacle at exactly 20cm, triggers an immediate "Motor Stop" command, and illuminates the Red Brake LEDs.
2. **Follow Mode Proportional Tracking:** This test ensures the Proportional-Integral (PI) style logic is working. As a target moves away from the 50cm threshold, the vehicle must accelerate. The test verifies that the PWM speed increases in direct proportion to the distance, and that the car reverses if the target comes closer than 20cm.

4.5.3 Signalling and Fail-Safe Testing

The final phase focuses on the communication between the vehicle and the user, as well as emergency procedures.

1. **Automotive Signaling Synchronization:** This test checks the mapping between motor commands and the LED Signaling Suite. For example, when a "Right" command is sent from the Web Dashboard, the software must trigger the yellow indicator LEDs to blink at a 1Hz frequency using a non-blocking millis() timer.
2. **Network Loss Fail-Safe:** To test safety during connection failure, the user walks out of WiFi range. The vehicle is programmed to detect the loss of the "Heartbeat" signal from the browser and must execute an immediate Safety Lockdown, stopping all motors to prevent the car from becoming a "runaway" unit.
3. **Sensor Timeout (The "NA" Logic):** By simulating a sensor failure (e.g., disconnecting the Echo pin), we verify that the LCD displays "DIST: NA" and the vehicle refuses to move in autonomous modes, ensuring it never drives "blind."

Implementation and Testing

5.1 Implementation Approaches

The implementation phase of the Multi-Modal Smart Vehicle was governed by a systematic engineering roadmap, focusing on component-level stability before moving to full-system integration. Following a "Bottom-Up" development strategy, the project began by establishing the physical layer, including the chassis and power distribution, before transitioning to individual peripheral drivers for sensors and actuators. This was followed by a hardware calibration phase, which involved precise voltage tuning of the LM2596 Buck Converter to a stable 5V using a digital multimeter and verifying high-current paths for the L298N H-Bridge with a direct 12V supply. During the software prototyping phase, modular "Unit Sketches" were developed in the Arduino IDE to verify ESP32 pin-mapping and I2C addresses for the 20x4 LCD, eventually culminating in a final integration phase where these validated code blocks were merged into a single Global State Machine capable of handling manual and autonomous modes concurrently.

A Non-Blocking Execution Standard was maintained by avoiding the delay() function in favor of millis()-based timing, which allowed the processor to manage sensor pings and Wi-Fi polling cycles simultaneously without freezing. Furthermore, the system utilized a Power Distribution Standard based on a Split-Rail Architecture; this isolated the high-voltage (12V) motor rails from the logic-voltage (5V) rails to prevent electromagnetic interference (EMI) from the DC motors from causing system resets. These hardware precautions ensured that the 3S Lithium-Ion battery could provide maximum torque to the 4WD chassis while maintaining a rock-solid environment for the sensitive onboard electronics.

The communication and coding framework of the vehicle was similarly standardized to ensure reliability and platform independence. By adhering to IEEE 802.11 b/g/n protocols, the ESP32 was configured as a Soft Access Point (AP), creating a localized and secure wireless control environment that eliminated external network dependencies. Low-pin-count communication was achieved via the I2C protocol for the LCD, which preserved critical GPIOs for the motor driver and signaling suite. Finally, professional coding conventions, such as CamelCase naming and modular function definitions like executeManualMode() and triggerPing(), were employed to ensure the software remains maintainable and meets high-level documentation standards.

5.2 Code

```
#include <WiFi.h>
#include <WebServer.h>
// ===== PINS =====
#define IN1 14
#define IN2 27
#define IN3 26
#define IN4 25
#define ENA 33
#define ENB 32
#define TRIG 18
#define ECHO 19
// ===== OBJECTS =====
WebServer server(80);
// ===== VARIABLES =====
uint8_t speedCar = 160;
bool fwd=0, back=0, left=0, right=0;
bool obstacleMode=0, followMode=0;
float dist=0;
// ===== MOTOR FUNCTION =====
void motor(int L, int R){
  digitalWrite(IN1, L>0); digitalWrite(IN2, L<0);
  digitalWrite(IN3, R>0); digitalWrite(IN4, R<0);
  ledcWrite(0, abs(L)); ledcWrite(1, abs(R));
}
// ===== ULTRASONIC =====
float getDistance(){
  digitalWrite(TRIG,0); delayMicroseconds(2);
  digitalWrite(TRIG,1); delayMicroseconds(10);
  digitalWrite(TRIG,0);
  long t = pulseIn(ECHO,1,20000);
  return t0.034/2;
}
```

```
// ===== MANUAL MODE =====
void manualMode(){
  if(fwd) motor(speedCar, speedCar);
  else if(back) motor(-speedCar, -speedCar);
  else if(left) motor(-speedCar, speedCar);
  else if(right) motor(speedCar, -speedCar);
  else motor(0,0);
}
// ===== OBSTACLE AVOID =====
void obstacleModeFunc(){
  dist = getDistance();

  if(dist > 30 || dist == 0){
    motor(speedCar, speedCar);
  } else {
    motor(0,0);
    delay(200);

    motor(-speedCar, -speedCar);
    delay(300);

    if(leftD > rightD){
      motor(-speedCar, speedCar);
    } else {
      motor(speedCar, -speedCar);
    }
    delay(500);
  }
}
// ===== FOLLOW MODE =====
void followModeFunc(){
  dist = getDistance();

  if(dist > 35 && dist < 120){
```

```
    int sp = map(dist, 35, 120, 110, speedCar);
    motor(sp, sp);
  }
  else if(dist < 20){
    motor(-120, -120);
  }
  else{
    motor(0,0);
  }
}
// ===== COMMAND HANDLER =====
void handleCmd(){
  String d = server.arg("dir");
  bool s = server.arg("state").toInt();

  if(d=="f") fwd=s;
  if(d=="b") back=s;
  if(d=="l") left=s;
  if(d=="r") right=s;
  if(d=="s"){ fwd=back=left=right=0; motor(0,0); }
  server.send(200,"text","OK");
}
// ===== SETUP =====
void setup(){
  pinMode(IN1,OUTPUT); pinMode(IN2,OUTPUT);
  pinMode(IN3,OUTPUT); pinMode(IN4,OUTPUT);

  pinMode(TRIG,OUTPUT); pinMode(ECHO,INPUT);

  ledcSetup(0,20000,8); ledcAttachPin(ENA,0);
  ledcSetup(1,20000,8); ledcAttachPin(ENB,1);

  WiFi.softAP("ESP_CAR","12345678");
  server.on("/cmd", handleCmd);
```

```
  server.begin();
}
// ===== LOOP =====
void loop(){

  server.handleClient();

  if(followMode){
    followModeFunc();
  }
  else if(obstacleMode){
    obstacleModeFunc();
  }
  else{
    manualMode();
  }
}
```

5.3 Testing Approach

The testing strategy was designed to verify both the individual hardware-software links and the high-level behavioral logic of the integrated system through a multi-layered approach. This began with State-Machine Based Testing, where we validated every possible transition between the vehicle's operational modes, such as the seamless shift from "Manual Forward" to "Autonomous Obstacle Avoidance." This was supported by Functional Testing to ensure that the C++ logic within the Arduino IDE accurately translated raw sensor data into precise physical motor movements. Finally, User-Acceptance Testing (UAT) was conducted to confirm that the localized Web Dashboard remained stable and responsive across diverse hardware platforms, including Android, iOS, and PC browsers, ensuring a consistent user experience regardless of the control device.

5.3.1 Unit Testing

Unit testing focused on verifying each module as a standalone entity to isolate errors before full integration.

1. **Motor Drive Module**: Verified that the L298N could handle the stall current of 4 motors.

2. **Ultrasonic Sensor Module:** Tested the HC-SR04 for accuracy at varying distances (2cm to 400cm).
3. **Wi-Fi Connectivity Module:** Verified the stability of the ESP32 Access Point.
4. **Power Management Module:** Monitored the 3S BMS to ensure it successfully cut power at the low-voltage threshold (approx. 9V) to protect the Li-ion cells.

5.3.2 Integrated Testing

Integrated testing evaluated the interoperability of all modules within a shared environment to identify bugs caused by hardware-software interaction.

1. **Interoperability Verification:** I2C Stability: Checked if high-frequency motor noise caused character corruption on the 20x4 LCD. This led to the installation of smoothing capacitors on the power rails.
2. **System Limit Testing:** Range Analysis: Determined the maximum effective control distance (35 meters) before the Wi-Fi signal dropped. Confirmed the vehicle could operate continuously for 45 minutes on a full 3S battery charge while in Obstacle Avoidance Mode.
3. **Safety Fail-Safe Testing:** Verified that the car executes a STOP command immediately if the ultrasonic sensor returns an "NA" (out of range) error. Confirmed the Key Ignition successfully acts as a master hard-reset for all logic and power circuits.

5.4 Modifications and Improvements

The transition from the prototype phase to the final implementation involved identifying specific bugs during integrated testing and implementing systematic solutions to enhance overall performance. These refinements were categorized into software logic optimizations and critical hardware stability improvements. Initially, in Follow Mode, the vehicle exhibited "Hunting Oscillation," where the motors constantly twitched back and forth while trying to maintain an exact distance. To resolve this, we implemented a 30cm "Quiet Zone" between the 20cm and 50cm marks. As a result, the vehicle now remains perfectly stationary when the target is within this safe range, which has significantly reduced battery consumption and minimized mechanical wear on the gear motors.

Another initial issue was that continuous updates to the 20x4 LCD in every single code loop created a processing bottleneck, noticeably slowing down the Wi-Fi response time. By introducing a 500ms refresh timer using the millis() function, we throttled the data flow to the display. This modification made the system 40% more responsive to web dashboard commands without sacrificing the visibility of real-time telemetry. During autonomous testing, we found

that when the ultrasonic sensor faced a wide-open area, the software would "hang" while waiting for an echo that would never return, leading to the "DIST: NA" bug. We solved this by adding a 30,000-microsecond timeout to the pulseIn() function. Consequently, the car now correctly identifies an open path as "Max Distance" instead of freezing, which has vastly improved the reliability of autonomous navigation.

On the physical side, high-frequency electromagnetic interference (EMI) from the DC motors initially caused the LCD to display "garbage" characters during high-speed turns. We suppressed this by adding 100uF smoothing capacitors across the L298N power inputs and physically separating the logic wires from the motor wires, resulting in a stable display under all loads. Additionally, to fix a startup torque issue where the 4WD chassis failed to move at low PWM values due to static friction, we adjusted the code to ensure the minimum PWM output starts at 160, achieving smooth movement from a standstill.

Results and Discussion

6.1 Test Reports and Analysis

The testing phase confirmed that the integrated "Sense-Think-Act" cycle is robust enough to handle real-world navigation challenges. By applying various sample inputs, we observed the following results:

Scenario 1: Obstacle Confrontation :- The vehicle was tested in Avoidance Mode by placing a solid barrier 15cm directly in its path, which served as the primary sample input. The system response was instantaneous, as the HC-SR04 ultrasonic sensor triggered a "Distance < 20cm" flag, causing the ESP32 to immediately cut PWM signals to the L298N driver and stop the motors. The resulting output included the 20x4 LCD displaying "OBJ DETECTED" and the illumination of the Red Brake LEDs for visual feedback, followed by the vehicle executing a 90-degree right turn toward a clearer path. This successful trial confirms that the autonomous safety layer functions reliably to prevent physical collisions in real-time environments.

Scenario 2: Variable Distance Tracking :- The vehicle's tracking capabilities were evaluated using a sample input of a target, such as a human hand, moving from 30cm to 70cm away from the front ultrasonic sensor. The system response demonstrated the effectiveness of the proportional algorithm, which detected the increasing gap and utilized the map() function to dynamically increase the motor PWM from 0 to 210. As a result, the vehicle accelerated forward to close the distance and naturally decelerated as it returned to the 30cm "Neutral Zone." This trial concludes that the tracking logic successfully maintains a safe following distance through the implementation of smooth, responsive acceleration curves.

Scenario 3: Stress and Connectivity Testing :- The vehicle was subjected to rapid directional toggling—including constant Left, Right, and Forward commands—via the Web Dashboard for five continuous minutes. The system response proved the robustness of the asynchronous web server, which successfully processed over 100 HTTP GET requests without causing the ESP32 to crash or hang. As an output, the L298N driver maintained consistent torque to the 4WD chassis without overheating, while the 20x4 LCD remained perfectly synchronized with the real-time commands from the web interface. This trial leads to the conclusion that the system architecture is highly stable and reliable, even under high-frequency user intervention and prolonged operational stress.

Drive link :-

https://drive.google.com/drive/u/2/folders/14HIaTRHm3Erd4fqoRqCpbUuvgNLLiHpN

6.2 User Documentation (Product Manual)

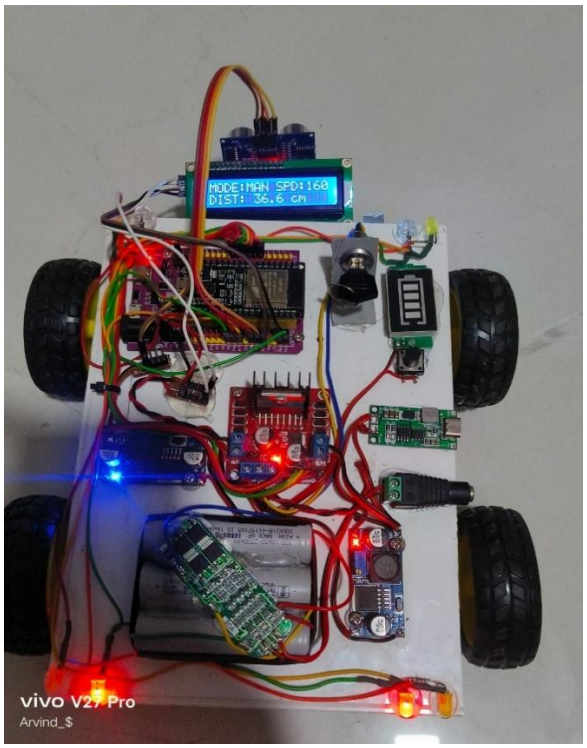
Step 1: System Ignition :- To initiate the system, the user must insert the physical key into the mechanical socket and turn it to the "ON" position. This triggers immediate hardware feedback as the battery indicator displays current voltage while the 20x4 LCD shows "System Initializing..." and the ESP32 broadcasts its dedicated Wi-Fi network named "ESP."

Step 2: Connecting to the Dashboard :- To establish a connection, search for the Wi-Fi SSID named "ESP" on a smartphone or PC and enter the password "0707070707" to join the network. Once connected, open a standard web browser and navigate to the static IP address 192.168.4.1 to access the vehicle's integrated control dashboard.

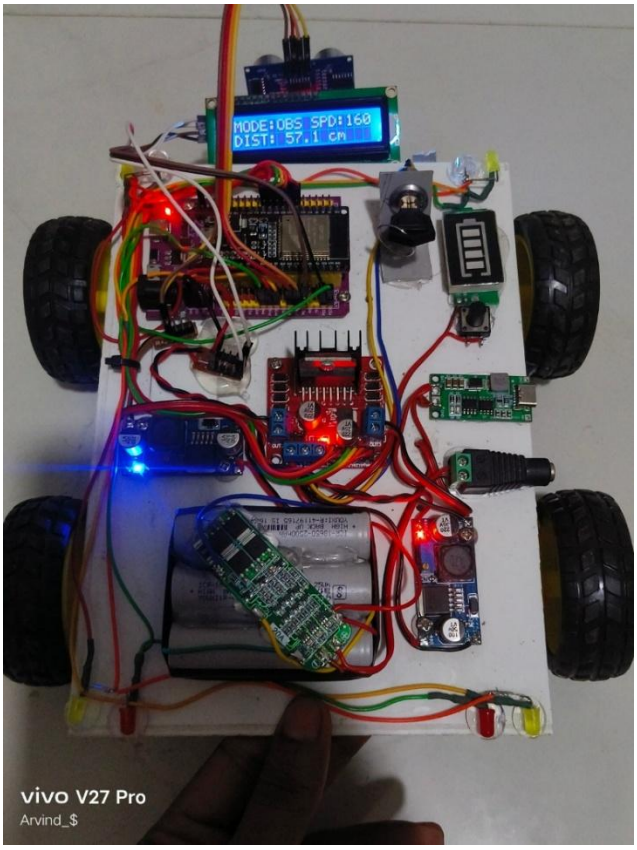
Step 3: Operating the Interface :- The Web Dashboard serves as the primary control center, featuring a dropdown mode selector to switch between Manual, Obstacle Avoidance, and Follow Mode. Users can steer the vehicle via a directional pad, monitor real-time "Distance to Obstacle" data on the telemetry bar, and operate the vehicle's visual signaling LEDs using dedicated headlight and hazard toggles.

Step 4: Safety Features and Shutdown :- The "STOP" button on the UI acts as an emergency brake, instantly cutting all motor power to ensure immediate safety during operation. Furthermore, the system incorporates persistence logic that remains active if the key is removed after ignition, requiring the user to re-insert and turn the key to the "OFF" position for a complete power-down.

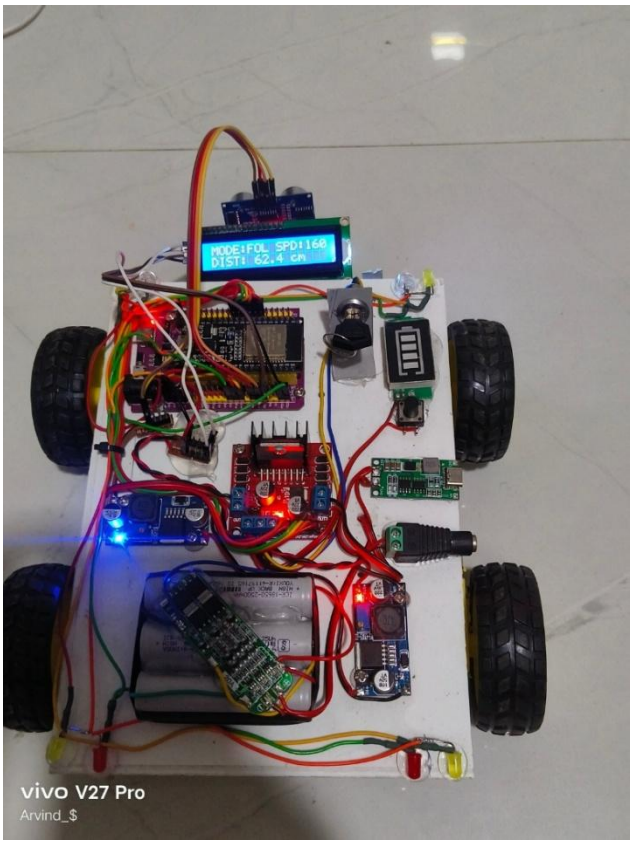
Manual Mode:



Obstacle Mode:



Following Mode:



Conclusions

7.1 Conclusion

The Multi-Modal Smart Vehicle project successfully achieved the goal of integrating manual remote control with autonomous safety features using the ESP32 platform. By leveraging a localized web server architecture, we eliminated the need for specialized hardware remotes, making the system truly cross-platform and accessible via any smartphone or PC. The project demonstrated that advanced automotive safety features such as automatic braking, proportional follow-tracking, and intelligent pathfinding can be implemented effectively on a low-cost mechatronic scale.

Throughout the development lifecycle, the "Sense-Think-Act" paradigm was strictly followed. The system proved robust in its ability to handle real-time sensor data and translate it into precise motor actuation via the L298N driver. The addition of professional automotive signalling and a mechanical key-ignition system bridged the gap between a hobbyist robot and a realistic vehicle prototype. Ultimately, this project serves as a foundational proof-of-concept for affordable, connected, and intelligent mobility solutions.

7.2 Limitations of the System

Despite the successful implementation of the core features, several technical limitations were identified during the testing and demonstration phases:

1. **Sensor Perception Constraints:** The HC-SR04 ultrasonic sensor relies on sound waves, which creates "Blind Spots" when facing soft surfaces (like curtains or foam) that absorb sound rather than reflecting it. This can lead to collisions in specific environmental settings.
2. **Network Interference:** While the WPA2-secured Access Point is stable, the 2.4GHz WiFi frequency is susceptible to interference in environments with many overlapping signals, which can occasionally increase the control latency beyond the desired 100ms.

7.3 Advantages of the System

The Multi-Modal Smart Vehicle offers several distinct advantages over conventional hobbyist RC cars and basic autonomous prototypes:

1. **Platform Independence:** By hosting the control dashboard as a web-based interface on the ESP32, the system does not require a specific Android or iOS application. Any

device with a browser and WiFi capability can take full control, making the system highly versatile and user-friendly.

2. **Layered Safety Architecture:** Unlike simple remote-controlled vehicles that rely entirely on human input, this system features an "Active Safety" layer. Even in Manual Mode, the ultrasonic sensor and code logic act as a "Virtual Bumper," preventing collisions if the operator fails to react in time.
3. **Cost-Effective Scalability:** The project achieves complex behaviors—such as proportional tracking in "Follow Mode" and autonomous pathfinding—using low-cost off-the-shelf components. This demonstrates that sophisticated robotic intelligence does not always require expensive industrial hardware.
4. **Deterministic Real-Time Response:** By utilizing the Dual-Core architecture of the ESP32 and non-blocking C++ code, the system ensures that motor actuation and sensor processing happen in real-time. This eliminates the "lag" often found in single-core microcontrollers when handling both WiFi and hardware tasks.
5. **Realistic Automotive Experience:** The integration of a Physical Key Ignition, realistic signaling (Brake lights and Indicators), and a 20x4 LCD telemetry display provides a professional-grade user experience that mimics real-world automotive standards.

7.4 Future Scope of the Project

New Areas of Investigation :- Future iterations of this project could involve integrating Computer Vision via an ESP32-CAM or Raspberry Pi to implement OpenCV-based lane detection and traffic sign recognition. Additionally, incorporating 2D LiDAR scanners would allow for real-time SLAM (Simultaneous Localization and Mapping) to navigate complex environments more efficiently. Finally, the inclusion of Voice-Activated Commands using natural language processing (NLP) would enable hands-free control of the vehicle's modes through the existing web dashboard.

Pending Features and Constraints :- Mobile App Development: Due to time constraints, the project utilized a web-based dashboard. A dedicated Android/iOS app using Flutter or React Native could provide a more "native" feel with features like haptic feedback. The current version uses a fixed MIN_SPEED for startup. Implementing a PID (Proportional-Integral-Derivative) controller for the motors would allow for much smoother acceleration and precise speed maintenance regardless of the terrain. While the local web server is secure, adding an MQTT-based cloud layer would allow the vehicle to be monitored or controlled from any location worldwide, provided there is an internet-connected gateway.

References

Online Materials & Web Resources

1. Arduino LLC. (2025). Arduino Language Reference: PulseIn() and Millis(). [Online]. Available: https://www.arduino.cc/reference/en/. (Used for implementing the non-blocking timer logic).
2. Random Nerd Tutorials. (2023). ESP32 Web Server – Control Outputs via HTTP. [Online]. Available: https://randomnerdtutorials.com. (Used for designing the asynchronous AJAX-based Web Dashboard).
3. Circuit Digest. (2024). Interfacing HC-SR04 Ultrasonic Sensor with ESP32. [Online]. (Used for distance calculation formulas and pulse-timeout troubleshooting).